1: Kubernetes primer

This chapter is split into two main sections.

- Kubernetes background – where it came from etc.
- Kubernetes as the Operating System of the cloud

Kubernetes background

Kubernetes is an *application orchestrator*. For the most part, it orchestrates containerized cloud-native microservices apps. How about that for a sentence full of buzzwords!

You'll come across terms like this a lot, so let's take a minute to explain what each of them means.

What is an orchestrator

An *orchestrator* is a system that *deploys* and *manages* applications. It can deploy your applications and dynamically respond to changes. For example, Kubernetes can:

- Deploy your application
- Scale it up and down dynamically based on demand
- Self-heal it when things break
- Perform zero-downtime rolling updates and rollbacks
- Lots more…

And the best part about Kubernetes… it does all of this without you having to supervise or get involved. Obviously, you have to set things up in the first place, but once you've done that, you sit back and let Kubernetes work its magic.

What is a containerised app

A *containerized application* is an app that runs in a container.

Before we had containers, applications ran on physical servers or in virtual machines. Containers are just the next iteration of how we package and run apps. As such, they're faster, more lightweight, and more suited to modern business requirements than servers and virtual machines.

Think of it this way.

- Apps ran on physical servers in the open-systems era (1980s and 1990s)
- Apps ran in virtual machines in the virtualisation era (2000s and into the 2010s)
- Apps run in containers in the cloud-native era (now)

While Kubernetes can orchestrate other workloads, such as virtual machines and serverless functions, it's most commonly used to orchestrate containerised apps.

What is a cloud-native app

A *cloud-native application* is one that's designed to meet cloud-like demands of auto-scaling, self-healing, rolling updates, rollbacks and more.

It's important to be clear that cloud-native apps are not applications that will only run in the public cloud. Yes, they absolutely can run on public clouds, but they can also run anywhere that you have Kubernetes, even your on-premises datacenter.

So, *cloud-native* is about the way applications behave and react to events.

What is a microservices app

A *microservices app* is built from lots of independent small specialised parts that work together to form a meaningful application. For example, you might have an e-commerce app that comprises all of the following small specialised components:

- Web front-end
- Catalog service
- Shopping cart
- Authentication service
- Logging service
- Persistent store

Each of these individual services is called a ***microservice***. Typically, each is coded and owned by a different team. Each can have its own release cycle and can be scaled independently. For example, you can patch and scale the logging *microservice* without affecting any of the others.

Building applications this way is vital for cloud-native features.

For the most part, each microservice runs as a container. Assuming this e-commerce app with the 6 microservices, there'd be one or more web front-end containers, one or more catalog containers, one or more shopping cart containers etc.

With all of this in mind, let's re-phrase that definition that was full of buzzwords…

Kubernetes deploys and manages (orchestrates) applications that are packaged and run as containers (containerized) and that are built in ways (cloud-native microservices) that allow them to scale, self-heal, and be updated in-line with modern cloud-like requirements.

We'll talk about these concepts a lot throughout the book, but for now, this should help you understand some of the main industry buzzwords.

Where did Kubernetes come from

Let's start at the beginning…

Amazon Web Services (AWS) changed the world when it brought us modern cloud computing. Since then, everyone else has been playing catch-up.

One of the companies trying to catch-up was Google. Google has its own very good cloud, and needs a way to abstract the value of AWS, **and** make it easier for potential customers to get off AWS and onto their cloud.

Google also has a lot of experience working with containers at scale. For example, huge Google applications, such as Search and Gmail, have been running at extreme scale on containers for a lot of years – since way before Docker brought us easy-to-use containers. To orchestrate and manage these containerised apps, Google had a couple of in-house proprietary systems called Borg and Omega.

Well, Google took the lessons learned from these in-house systems, and created a new platform called Kubernetes, and donated it to the newly formed Cloud Native Computing Foundation (CNCF) in 2014 as an open-source project.

CLOUD NATIVE
COMPUTING FOUNDATION

https://www.cncf.io

Figure 1.1

Kubernetes enables two things Google and the rest of the industry needs.

1. It abstracts underlying infrastructure such as AWS
2. It makes it easy to move applications on and off clouds

Since its introduction in 2014, Kubernetes has become the most important cloud-native technology on the planet.

Like many of the modern cloud-native projects, it's written in Go (Golang), it's built in the open on GitHub (at `kubernetes/kubernetes`), it's actively discussed on the IRC channels, you can follow it on Twitter (@kubernetesio), and `slack.k8s.io` is a pretty good slack channel. There are also regular meetups and conferences all over the planet.

Kubernetes and Docker

Kubernetes and Docker are complementary technologies.

Docker has tools that build and package applications as container images. It can also run containers. Kubernetes can't do either of those things. Instead, Kubernetes operates at a higher level providing orchestration services such as self-healing, scaling and updates.

It's common practice to use Docker for build-time tasks such as packaging apps as containers, but then use a combination of Kubernetes and Docker to run them. In this model, Kubernetes performs high-level orchestration tasks such as self-healing, scaling and rolling updates, but it needs a tool like Docker to perform low-level tasks such as starting and stopping containers.

Assume you have a Kubernetes cluster with 10 nodes to run your production applications. Behind the scenes, each cluster node is running Docker as its *container runtime*. This means Docker is the low-level technology that starts and stops the containerised applications. Kubernetes is the higher-level technology that looks after the bigger picture, such as deciding which nodes to run containers on, deciding when to scale up or down, and executing updates.

Figure 1.2 shows a simple Kubernetes cluster with *some* nodes using Docker as the container runtime.

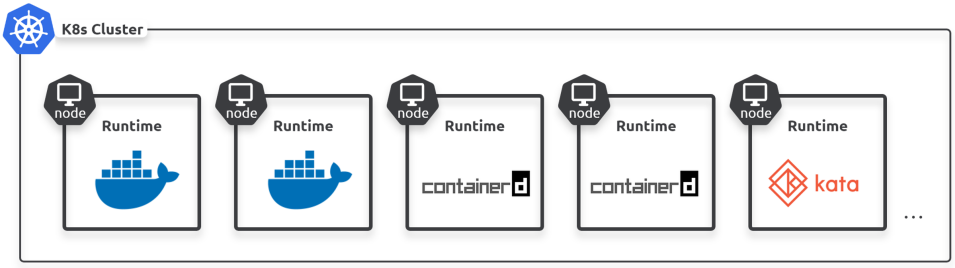

Figure 1.2

As can be seen, Docker isn't the only *container runtime* Kubernetes supports. In fact, Kubernetes has a couple of features that abstract the container runtime and make it interchangeable:

1. The *Container Runtime Interface (CRI)* is an abstraction layer that standardizes the way 3rd-party container runtimes work with Kubernetes.
2. *Runtime Classes* allows you to create different *classes* of runtimes. For example, the *gVisor* or *Kata Containers* runtimes might provide better workload isolation than the *Docker* and *containerd* runtimes.

Kubernetes 1.20 deprecated Docker as a *runtime*. Container *images* created by Docker will continue to work as normal, and this won't change. But a future release of Kubernetes will end support of Docker **as a runtime**. To streamline the deprecation process, many Kubernetes clusters already ship with `containerd` as the default runtime. `containerd` is effectively a stripped-down version of Docker with just the stuff that Kubernetes needs. It's pronounced *container dee* and is a strategic Kubernetes container runtime.

While all of this is interesting, it's low-level stuff that shouldn't impact your Kubernetes learning experience. For example, whichever container runtime you use, the regular Kubernetes commands and patterns will continue to work as normal.

What about Kubernetes vs Docker Swarm

In 2016 and 2017 we had the *orchestrator wars* where Docker Swarm, Mesosphere DCOS, and Kubernetes competed to become the de-facto container orchestrator. To cut a long story short, Kubernetes won.

However, Docker Swarm is still under active development and is popular with small companies that need a simple alternative to Kubernetes.

Kubernetes and Borg: Resistance is futile!

There's a good chance you'll hear people talk about how Kubernetes relates to Google's *Borg* and *Omega* systems.

As previously mentioned, Google has been running containers at scale for a long time – apparently crunching through billions of containers a week. So yes, Google has been running things like *search*, *Gmail*, and *GFS* on **lots** of containers for a very long time.

Orchestrating these containerised apps was the job of a couple of in-house technologies called *Borg* and *Omega*. So, it's not a huge stretch to make the connection with Kubernetes – all three are in the game of orchestrating containers at scale, and they're all related to Google.

However, it's important to understand that Kubernetes **is not** an open-sourced version of *Borg* or *Omega*. It's more like Kubernetes shares its DNA and family history with them.

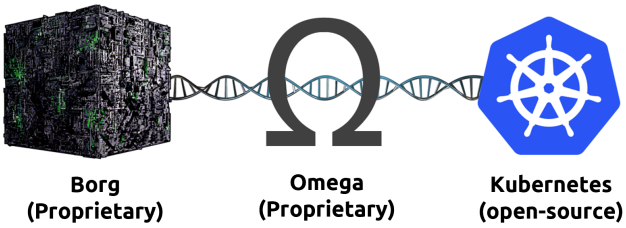

Figure 1.3 - Shared DNA

The point is, all three are separate, but all three are related. In fact, some of the people who built Borg and Omega were, and still are, involved with Kubernetes. So, although Kubernetes was built from scratch, it leverages much of what was learned at Google with Borg and Omega.

As things stand, Kubernetes is an open-source project donated to the CNCF in 2014. It's licensed under the Apache 2.0 license, version 1.0 shipped way back in July 2015, and at-the-time-of-writing, we're already into the 1.20's.

Kubernetes – what's in the name

The name **Kubernetes** (koo-ber-net-eez) comes from the Greek word meaning *Helmsman* – the person who steers a seafaring ship. This theme is reflected in the logo, which is the wheel (helm control) of a ship.

Figure 1.4 - The Kubernetes logo

Some of the people involved in the creation of Kubernetes wanted to call it *Seven of Nine*. If you know your Star Trek, you'll know that *Seven of Nine* is a **Borg** drone rescued by the crew of the USS Voyager under the command of Captain Kathryn Janeway. Sadly, copyright laws prevented it from being called *Seven of Nine*. So the creators gave the logo seven spokes as a tip-of-the-hat to *Seven* of Nine.

One last thing about the name before moving on. You'll often see it shortened to "K8s" (pronounced "kates"). The number 8 replaces the 8 characters between the "K" and the "s" and is why people sometimes joke that Kubernetes has a girlfriend called Kate.

Kubernetes as the operating system of the cloud

Kubernetes has emerged as the de facto platform for deploying and managing cloud-native applications. In many ways, it's like an operating system (OS) for the cloud. Consider this:

- You install a traditional OS (Linux or Windows) on a server, and it *abstracts* server resources and *schedules* application processes
- You install Kubernetes on a cloud, and it *abstracts* cloud resources and *schedules* application microservices

In the same way that Linux *abstracts* the hardware differences between server platforms, Kubernetes *abstracts* the differences between different private and public clouds. Net result… as long as you're running Kubernetes, it doesn't matter if the underlying systems are on premises in your own datacenter, edge devices, or in the public cloud.

With this in mind, Kubernetes is a major step towards a true *hybrid cloud*, allowing you to seamlessly move and balance workloads across multiple different public and private cloud infrastructures. You can also migrate *to* and *from* different clouds, meaning you can choose a cloud today and not have to stick with that decision for the rest of your life.

Cloud scale

Generally speaking, cloud-native microservices applications make our previous scalability and complexity challenges look easy – we've just said that Google goes through billions of containers every week!

That's great, but most of us are nothing like Google. What about the rest of us?

Well… as a general rule, if your legacy apps have hundreds of VMs, there's a good chance your containerized cloud-native microservices apps will have thousands of containers. With this in mind, you'll need help managing them.

Say hello to Kubernetes.

Also, we live in a business and technology world that's increasingly fragmented and constantly in a state of disruption. With this in mind, we desperately need a framework and platform that is widely accepted and hides complexity.

Again, say hello to Kubernetes.

Application scheduling

A typical computer is a collection of CPU, memory, storage, and networking. But modern operating systems have done a great job abstracting that. For example, how many developers care which CPU core or exact memory address their application uses? Not many, we let the OS take care of things like that. And it's a good thing, making the world of application development a far friendlier place.

Kubernetes does a similar thing with cloud and datacenter resources. At a high-level, a cloud or datacenter is a pool of compute, network and storage resources. Kubernetes abstracts them, meaning you don't have to hard code which node or storage volume your applications run on, you don't even have to care which cloud they run on. Kubernetes takes care of all that.

So, gone are the days of naming your servers, mapping storage volumes in a spreadsheet, and otherwise treating your infrastructure assets like *pets*. Modern cloud-native apps don't usually care. In the cloud-native world, we just say *"Hey Kubernetes, here's an app. Please deploy it and make sure it keeps running…"*.

A quick analogy...

Consider the process of sending goods via a courier service.

You package the goods in the courier's standard packaging, slap one of their labels on it, and hand it over to the courier. The courier is responsible for everything else. This includes all the complex logistics of which planes and trucks it goes on, which highways to use, and who the drivers should be etc. They also provide services that let you do things like track your package and make delivery changes. The point is, the only thing *you* have to do is package and label the goods. The courier does everything else.

It's the same for apps on Kubernetes. You package the app as a container, give it a Kubernetes manifest, and let Kubernetes take care of deploying it and keeping it running. You also get a rich set of tools and APIs that let you introspect (observe and examine) it. It's a beautiful thing.

Chapter summary

Kubernetes was created by Google based on lessons learned running containers at scale for a lot of years. It was donated to the community as an open-source project and is now the industry standard API for deploying and managing cloud-native applications. It runs on any cloud or on-premises datacenter and abstracts the underlying infrastructure. This allows you to build hybrid clouds, as well as migrate on and off the cloud and between different clouds. It's open-sourced under the Apache 2.0 license and lives within the Cloud Native Computing Foundation (CNCF).

Tip!

Kubernetes is a fast-moving project under active development. But don't let this put you off – embrace it. Change is the new normal.

To help you keep up to date, feel free to follow me and subscribe to my newsletter and YouTube channel.

- nigelpoulton.com
- twitter.com/nigelpoulton
- linkedin.com/in/nigelpoulton/
- youtube.com/nigelpoulton